# Programación III

| | |
|---|---|
| Comenzado el | jueves, 24 de junio de 2021, 11:00 |
| Estado | Finalizado |
| Finalizado en | jueves, 24 de junio de 2021, 11:52 |
| Tiempo empleado | 52 minutos 5 segundos |
| Puntos | 16,50/22,00 |
| Calificación | **7,50** de 10,00 (**75**%) |

Pregunta **1**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

## En el patrón MVC, la Vista es la encargada de gestionar los eventos producidos por los componentes activos (botones, enlaces, etc).

```
Correcta: 1
Incorrecta: -0,5
```

○ a. V

● b. F ✓

Respuesta correcta

La respuesta correcta es:
F

Pregunta **2**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

## Considerando las siguientes implementaciones del método equals

```
Elija una sola respuesta
Correcta: 1
Incorrecta: -0,5
```

A)

```
public class Primero
{
    private String nombre;
    private long dni; // identificador que no se repite
    private Date fecha_nac;
    @Override
    public boolean equals(Object object)
    {
        if (this == object)
        {
            return true;
        }
        if (!(object instanceof Primero))
        {
            return false;
        }
        final Primero other = (Primero) object;
        if (!(nombre == null ? other.nombre == null : nombre.equals(other.nombre)))
        {
            return false;
        }
        if (dni != other.dni)
        {
            return false;
        }
        if (!(fecha_nac == null ? other.fecha_nac == null : fecha_nac.equals(other.fecha
        {
            return false;
        }
        return true;
    }
}
```

B)

```
public class Primero
{
    private String nombre;
    private long dni; //no se repite
    private Date fecha_nac;
    @Override
    public boolean equals(Object object)
    {
        if (this == object)
        {
            return true;
        }
        if (!(object instanceof Primero))
        {
            return false;
        }
        final Primero other = (Primero) object;
        if (dni != other.dni)
        {
            return false;
        }
        return true;
    }
}
```

Seleccione una:

● a. la B) admite que dos objetos sean iguales teniendo el mismo dni pero tal vez diferentes nombre y fecha_nac. ✓ Verdadero

○ b. la A) determina que dos objetos SÍ son iguales teniendo al menos el mismo dni.

○ c. Conviene usar la A) al momento de determinar la igualdad entre dos objetos, conociendo en uno de ellos, solamente uno de los tres atributos.

○ d. La B) representa una situación más real. Es suficiente utilizar el atributo dni para determinar la igualdad de dos objetos dentro de este dominio.

Respuesta correcta

Las respuestas correctas son: la B) admite que dos objetos sean iguales teniendo el mismo dni pero tal vez diferentes nombre y fecha_nac., La B) representa una situación más real. Es suficiente utilizar el atributo dni para determinar la igualdad de dos objetos dentro de este dominio.

Pregunta **3**
Finalizado
Puntúa 3,00 sobre 3,00
Marcar pregunta

## Explique la diferencia entre synchronized y wait().

Para qué sirve cada uno. Explique también para qué se usa la *condición* y por qué va dentro de un while.

```
public synchronized void metodo()
{
    while( //condición... )
    {
        wait();
    }
}
```

Establezca un ejemplo (un problema), relatelo muy brevemente y determine para qué se utilizaría synchronized y para qué se utilizaría wait().

El hecho que un método sea synchronized determina que al momento de que un hilo de ejecución invoca a ese método la instancia y sus atributos quedan bloqueados, lo que quiere decir que ningún otro método va a poder acceder a estos. Esto se hace para no generar inconsistencias entre los diferentes hilos de ejecución.

Por otro lado el wait() se ejecuta dentro de un método sincronizado y se utiliza principalmente para indicarle a un hilo que "no es su momento" de ser ejecutado. Cuando se ejecuta esta sentencia, todos los recursos bloqueados por el hilo son liberados, y otro hilo puede acceder a estos. El hilo que ejecuto un wait() reanudara su ejecución al recibir un notify() o un notiffyAll()

La condición se utiliza para indicarle a un hilo cuando puede salir de su estado de letargo. Al momento de despertar, volverá a evaluar la condición del while, y de no cumplirla, se ejecutara de nuevo el wait(). Se incluye dentro de un while ya que si estuviera dentro de un if() continuaría su ejecución luego del primer notifyAll(), sin chequear si se cumple la condición para q este hilo pueda realizar su función.

Por ejemplo, si tuviera un problema en el cual quisiera meter un numero dentro de una caja, tendria hilos que ponen un numero y otros que lo sacan. El metodo de sacar y poner deberían ser sincronizado para que todos los hilos tuvieran la misma información al momento de ejecutarse. Y los wait() de los hilos que se encargan de poner numeros seria

while(numero!=null)

wait();

Mientras que los que sacan el numero seria

while(numero==null)

wait();

Comentario:

Pregunta **4**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

## En el Patrón MVC. ¿De quién es la responsabilidad de validar el formato correcto de los datos que el usuario ingresa?

Seleccione una:

● a. Vista ✓

○ b. Controlador

○ c. Modelo

Respuesta correcta

La respuesta correcta es: Vista

Pregunta **5**
Incorrecta
Puntúa -0,50 sobre 1,00
Marcar pregunta

## Un objeto de tipo Observer sólo puede observar a un objeto de tipo Observable

```
Correcta: 1
Incorrecta: -1
```

○ a. F

● b. V ✗

Respuesta incorrecta.

La respuesta correcta es:
F

Pregunta **6**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

## Elija la/s respuesta/s correctas. Una elección equivocada tiene penalización. El patrón State permite:

```
Correcta vale 1
Incorrecta vale -0,5
```

Seleccione una:

○ a. Determinar un algoritmo general en la clase base y determinar los comportamientos particulares a través del mecanismo de herencia

● b. Permitir que un objeto cambie el comportamiento al cambiar el estado ✓

○ c. Permite añadir responsabilidades a un objeto en tiempo de ejecución

Respuesta correcta

Permitir que un objeto cambie el comportamiento al cambiar el estado

La respuesta correcta es: Permitir que un objeto cambie el comportamiento al cambiar el estado

Pregunta **7**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

Dada la clase genérica Caja<T>, siendo T el identificador del tipo de datos parametrizado, determine el valor de verdad de la variable rta:

```
Caja<Juguete> cajaJuguete = new Caja<Juguete>();
Caja<Herramienta> cajaH = new Caja<Herramienta>();
boolean rta = cajaJuguete.getClass() != cajaHerramienta().getClass();
```

Seleccione una:

○ Verdadero

● Falso ✓

Todas la invocaciones de clases genéricas son expresiones de una clase.

Al instanciar una clase genérica no se crea una nueva clase.

La respuesta correcta es 'Falso'

Pregunta **8**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

### Se tiene una clase Docente, que se extiende de Persona.

## Se necesita que trabaje en forma concurrente (solamente la clase Docente). Para lograr ésto y no perder la línea de herencia ....

Elija todas las opciones correctas.

La opciones incorrectas elegidas como correctas tienen penalización.

Seleccione una o más de una:

☑ a. Se debe implementar la interface Runnable en la clase Docente. ✓

☐ b. Se debe comenzar la ejecución del objeto de tipo Docente con la instrucción run()

☑ c. Se debe crear un nuevo objeto de tipo Thread. ✓

☐ d. Se debe implementar la interface Runnable en la clase Persona.

Respuesta correcta

Las respuestas correctas son: Se debe implementar la interface Runnable en la clase Docente., Se debe crear un nuevo objeto de tipo Thread.

Pregunta **9**
Incorrecta
Puntúa -0,50 sobre 1,00
Marcar pregunta

## El uso de tipos genéricos permite la comprobación de tipos correctos en tiempo de compilación.

```
Valor respuesta correcta: 1
Valor respuesta incorrecta: -0,5
```

● a. F ✗

○ b. V

Respuesta incorrecta.

La respuesta correcta es:
V

Pregunta **10**
Incorrecta
Puntúa -0,50 sobre 1,00
Marcar pregunta

## La serialización es una forma de hacer una clonación

● a. F ✗

○ b. V

Respuesta incorrecta.

La respuesta correcta es:
V

Pregunta **11**
Finalizado
Puntúa 2,00 sobre 3,00
Marcar pregunta

## De un ejemplo de un caso donde sea conveniente utilizar el patrón State.

El ejemplo no debe ser ninguno de los vistos en clase ni en el TPE, o sea, ni el del automóvil ni el de la ambulancia.
Bosqueje la solución haciendo un diagrama de clases completo (o escribiendo de las clases la jerarquia de herencia, los atributos y las cabeceras de métodos).
Establezca los atributos que determinarán los diferentes estados del objeto, determine los estados, los métodos, etc.
Solamente el diagrama de clases con atributos y cabecera de métodos.
No es necesario que haga un gráfico.

Se podria tener una pantalla la cual tiene 2 estados: prendida o apagada. Si esta apagada no realizara ninguna accion salvo encenderla, y si esta encendida se la puede apagar o se puede proyectar una imagen

```
Clase Pantalla{
  IState estado;
  public pantalla(){
    this.estado = new apagadaState(this);
}
  public void setState(IState estado){ this.estado = estado}
  public void encender(){ this.estado.encender}
  public void apagar(){ this.estado.apagar}
  public void proyectar(){ this.estado.proyectar}
}
```

La interfez IState va a tener 3 metodos: enceder, apagar y proyectar

2 Clases van a implentar IState: apagadaState y encendidaState

```
public interface IState{
   public void encender;
   public void apagar;
   public void proyectar;
}

public Class apagadaState implements IState{
    Pantalla p;

   public void encender{
       this.p.setState(new encendidoState());
}
   public void apagar{ }
   public void proyectar{ }
}

public Class encididaState implements IState{
    Pantalla p;

   public void encender{ }
   public void apagar( this.p.setState(new apagadaState());}
   public void proyectar{//codigo de proyeccion }
}
```

Comentario:
Te faltó agregar el parámetro de la pantalla en el constructor de las clases de estado.

Pregunta **12**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

## La función específica de la instrucción wait() es evitar que dos o más hilos accedan en forma simultánea a un recurso compartido

```
Correcta: 1
Incorrecta: -1
```

● a. F ✓

○ b. V

Respuesta correcta

La respuesta correcta es:
F

Pregunta **13**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

En el Patrón State existe una relación de Composición ente la clase de 'contexto' y las clases 'estado'.

Seleccione una:

● Verdadero ✓

○ Falso

La respuesta correcta es 'Verdadero'

Pregunta **14**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

Un método estático no puede ser sincronizado ya que el método start() de Thread es de instancia y no estático.

● a. F ✓

○ b. V

Respuesta correcta

La respuesta correcta es:
F

Pregunta **15**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

## En el patrón MVC, es conveniente que la clase vista cree el modelo y el controlador para poder dar inicio al programa.

```
Correcta: 1
Incorrecta: -0,5
```

● a. F ✓

○ b. V

Respuesta correcta

La respuesta correcta es:
F

Pregunta **16**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

## Un método sincronizado no puede invocar a otro método sincronizado ya que el invocado no podrá ejecutarse.

```
Correcta: 1
Incorrecta: -0,5
```

● a. F ✓

○ b. V

Respuesta correcta

La respuesta correcta es:
F

Pregunta **17**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

## Para poder persistir un objeto utilizando archivos XML es necesario

Seleccione una:

○ a. Que todos sus métodos sean públicos y sus atributos privados

○ b. Tener un constructor que nos permita pasar por prámetro los atributos que queremos persistir

● c. Tener un constructor público sin parámetros y getters y setters públicos ✓

○ d. Ninguna de las anteriores

Respuesta correcta

La respuesta correcta es: Tener un constructor público sin parámetros y getters y setters públicos

Pregunta **18**
Correcta
Puntúa 1,00 sobre 1,00
Marcar pregunta

## De acuerdo al siguiente enunciado elija las opciones correctas. Tenga en cuenta que existen dos clases principales que plantean dos formas diferentes de utilizar los métodos de ambos objetos

```
Valor respuesta correcta: 1
Valor respuesta incorrecta: -0,5
```

```
public class MiObserver implements Observer
{
    private MiObservable unObservable;
    //otros atributos

    public MiObserver()
    {
        //inicialización de otros atributos
    }

    public MiObserver(MiObservable unObservable)
    {
        this.setUnObservable(unObservable);
        this.getUnObservable().addObserver(this);
        //inicialización de otros atributos
    }

    @Override
    public void update(Observable o, Object arg)
    {
        // TODO Implement this method
    }

    public MiObservable getUnObservable()
    {
        return unObservable;
    }

    public void setUnObservable(MiObservable unObservable)
    {
        this.unObservable = unObservable;
    }
}

public class MiObservable extends Observable
{
    //atributos especificos

    public MiObservable()
    {
        super();
    }
}

public class Main
{
    public static void main(String args[])
    {
        MiObservable modelo;
        MiObserver ojo;
        modelo = new MiObservable();
        ojo = new MiObserver(modelo);
    }
}

public class Main2
{
    public static void main(String args[])
    {
        MiObservable modelo;
        MiObserver ojo;
        modelo = new MiObservable();
        ojo = new MiObserver();
        ojo.setUnObservable(modelo);
        modelo.addObserver(ojo);
    }
}
```

Seleccione una:

○ a. Main1 y Main2 tienen códigos correctos y el diseño de Main2 es mejor que el de Main1 debido a un tema de asignación de responsabilidades y prevención de errores

○ b. el código de la clase Main2 no funciona

● c. Main1 y Main2 tienen códigos correctos y el diseño de Main1 es mejor que el de Main2 debido a un tema de asignación de responsabilidades y prevención de errores ✓

Respuesta correcta

La respuesta correcta es: Main1 y Main2 tienen códigos correctos y el diseño de Main1 es mejor que el de Main2 debido a un tema de asignación de responsabilidades y prevención de errores

Finalizar revisión

◄ Ejemplo Persistencia cuando hay patron Singleton

Ir a...

Resultados del 2° Parcial ►

## Navegación por el cuestionario

1 ✓ 2 ✓ 3 ✓ 4 ✓ 5 6 ✓ 7 ✓ 8 ✓
9 10 11 12 ✓ 13 ✓ 14 ✓ 15 ✓ 16 ✓
17 ✓ 18 ✓

Mostrar una página cada vez

Finalizar revisión

Copyright © 2021 - DIFI - FI - UNMdP